

Dynamo

Key-value store – DynamoDB · Cassandra · Riak. ఇక్కడ scaling సులభం: joins లేవు, transactions లేవు, keys మధ్య క్రమం లేదు. **మొత్తం design బడ్జెట్ విఫలాలకి మరియు ఏకకాలానికి వెళ్తుంది.**

మూడు విరుపులు, మరియు ప్రతిదాని పరిష్కారానికి దాని సొంత విరుపు ఉంది. Consistent hashing కదలికని 94% నుంచి 1.4% కి తెస్తుంది – మరియు **34 రెట్లు అసమతుల్యత** తెస్తుంది. Last-write-wins మూడు ఏకకాల రాతల్లో **రెండింటిని** పోగొడుతుంది. Vector clocks దాన్ని సరిచేస్తాయి – మరియు **తొలగించిన వస్తువుని** తిరిగి తెస్తాయి.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node` lo run చేసినదే.

ఈ DOC యొక్క ఆకారం మిగతా వాటికంటే వేరు

మిగతా deep dives lo నమూనా ఇది: సహజమైన design → విరుపు → పరిష్కారం.

ఇక్కడ నమూనా ఇది: సహజమైన design → విరుపు → పరిష్కారం → ఆ పరిష్కారం యొక్క సొంత విరుపు → రెండో పరిష్కారం.

మూడుసార్లు అదే జరుగుతుంది, మరియు అది యాదృచ్ఛికం కాదు. ఈ problem lo ప్రతి ఎంపిక ఒక వ్యాపారం, ఒక మెరుగుదల కాదు – ఎందుకంటే మీరు పోరాడుతున్నది పరిమాణంతో కాదు, విఫలాలతో.

మరియు విఫలాలని "పరిష్కరించలేరు". మీరు వాటిని ఎక్కడికీ తరలించాలో మాత్రమే నిర్ణయించగలరు.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఏది సమస్య కాదు
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: keys ఎక్కడ ఉంటాయి

3. Step – `hash(key) % N`
4. మొదటి విరుపు – ఒక node కలిపితే 94.1% keys కదులుతాయి
5. Step – ring · మరియు ring యొక్క సొంత విరుపు
6. Step – virtual nodes

Part 3 – రెండో విరుపు: ఇద్దరు ఒకసారి రాస్తే

7. Step – రెండు clients, రెండు replicas
8. రెండో విరుపు – LWW మూడింట రెండు పోగొడుతుంది
9. Step – vector clocks · మరియు తిరిగి వచ్చే వస్తువు

Part 4 – మూడో విరుపు: quorum

10. Step – $R + W > N$
11. మూడో విరుపు – strict అయితే తిరస్కరణ, sloppy అయితే పాతది
12. Step – hinted handoff, read repair · మరియు అవి ఎంత సరిపోవు

Part 5 – నిరూపణ మరియు ప్రదర్శన

13. మొత్తం code · 1.9 లక్షల రాతలు · mutation testing

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి
15. నోటీస్ చెప్పాల్సిన English script
16. Follow-ups – Merkle trees, CRDTs, secondary indexes
17. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఏది సమస్య కాదు

ఒక key-value store. సెకనుకి కోటి ops. 100 TB. ఎప్పుడూ రాయగలగాలి – **shopping cart ఎప్పుడూ "ఇప్పుడు కుదరదు" అనకూడదు.**

సెకనుకి 10 మిలియన్ ops · 100 TB · 3 కాపీలు

ఒక node సెకనుకి 50,000 ops → 200 nodes
ఒక్కో node మీద data → 1536 GB

keys మధ్య joins లేవు, transactions లేవు, క్రమం లేదు.
→ nodes కలిపితే throughput సరళంగా పెరుగుతుంది. ఇది సులభమైన భాగం.

ఈ ఒక్క PROBLEM LO SCALING నిజంగా సులభం – మరియు అది ఒక సంతకం

HLD 12 lo ఒక document ని shard చేయలేము. HLD 13 lo partition సంఖ్య శాశ్వతమైనది. ఇక్కడ? **ఒక key ఇంకో key ని ఎప్పుడూ చూడదు.**

అంటే 200 nodes ని 400 చేస్తే throughput రెట్టింపు. ముగిసింది.

కాబట్టి ఒక ప్రశ్న అడగాలి: **ఈ problem lo కష్టమైనది ఏమిటి?** జవాబు – 200 machines ఉన్నప్పుడు **ఏదో ఒకటి ఎప్పుడూ పాడైపోయి ఉంటుంది.**

అదే అసలు లెక్క:

అసలు ప్రశ్న: ఒక key యొక్క 3 replicas lo ఎన్ని ఏ క్షణంలోనైనా అందుబాటులో ఉండవు?

ఒక node అందుబాటులో లేని సమయం | కనీసం 1 replica కింద | కనీసం 2 replicas కింద

0.1% (reboot, deploy)	0.30%	0.000%
0.5%	1.49%	0.007%
1% (GC, network)	2.97%	0.030%
5% (చెడ్డ రోజు)	14.26%	0.725%

ఈ రెండు నిలుపు వరుసలే మొత్తం DYNAMO PAPER

ఎడమవైపు – ఎంత తరచుగా మీ మూడు replicas లో ఒకటి లేదు. ఒక node 1% సమయం అందుబాటులో లేకపోతే, ప్రతి 34 requests లో ఒకదానికీ ఒక replica తక్కువ. ఇది అరుదు కాదు, సాధారణం.

కుడివైపు – ఎంత తరచుగా రెండు లేవు. అప్పుడు strict quorum ($W=2$) రాతని తిరస్కరించాలి.

"ఎప్పుడూ రాయగలగాలి" అనే అవసరం ఉంటే, మీరు ఎడమ నిలుపు వరుసతో జీవించాలి – దాన్ని తొలగించలేరు.

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	ఎందుకు అడుగుతున్నాం
రాత ఎప్పుడూ విజయవంతం కావాలా, లేక తిరస్కరించొచ్చా?	ఇదే §11 యొక్క మొత్తం నిర్ణయం
ఇద్దరు ఒకేసారి రాస్తే ఏమి జరగాలి?	§8. "తర్వాతిది గెలవాలి" అంటే మీరు డేటా పోగొట్టుకుంటున్నారు
ఒక పాత విలువ చదివితే ఫరవాలేదా – ఎంతసేపు?	§12 – read repair ఎంత ముఖ్యమో ఇది నిర్ణయిస్తుంది
Value ఒక set కా, ఒక counter కా, ఒక blob కా?	§9 – సరైన conflict resolution దీని మీద ఆధారపడుతుంది
Nodes ఎంత తరచుగా చేరతాయి/పోతాయి?	§4 – మరియు అది మీరు అనుకున్నదాని కంటే ఎక్కువ
Range queries కావాలా?	అవును అంటే hash కాదు, order-preserving partitioning – వేరే problem
ఒక key ఎంత వేడిగా ఉండగలదు?	ఒక వేడి key ఒక node ని ముంచేస్తుంది; vnodes సహాయపడవు
గడియారాలు ఎంత సమకాలీకృతం?	§8 – ± 50 ms తేడా అంటే 12% రాతలు తప్పు క్రమంలో

ఒక ప్రశ్న అడిగితే మీరు దీన్ని నిజంగా నడిపారని తెలుస్తుంది

"ఒక conflict వచ్చినప్పుడు – ఎవరు పరిష్కరిస్తారు? Database, లేక application?"

ఎందుకంటే §9 లో కొలుస్తాం: **database ఒక్కటే సరిగ్గా పరిష్కరించలేదు**. Vector clocks "ఇవి రెండూ ఏకకాలం" అని చెప్పగలవు – కానీ "అప్పుడు ఏమి చేయాలి" అని చెప్పలేవు.

ఒక shopping cart కి జవాబు "రెండింటినీ కలపు". ఒక counter కి "కూడు". ఒక profile పేరుకి "వాడుకరిని అడుగు". ఈ మూడూ వేర్వేరు, మరియు database కి తెలియదు.

Part 2 – మొదటి విరుపు: keys ఎక్కడ ఉంటాయి

3. Step – `hash(key) % N`

200 nodes. ఒక key ఏ node కి?

```
node = hash(key) % 200;
```

సమానంగా పంచుతుంది, లెక్క వేగవంతమైనది, ఏ metadata అవసరం లేదు. ఇది సరైన మొదలు.

ఇప్పుడు ఒక node కలుపుదాం – traffic పెరిగింది, లేదా ఒకటి చనిపోయి మార్చాలి.

4. మొదటి విరుపు – ఒక node కలిపితే 94.1% keys కదులుతాయి

2 లక్షల keys, 16 nodes నుంచి 17 కి:

1. ఒక node కలిపితే ఎన్ని keys కదులుతాయి?

విధానం	16 → 17 nodes ఆదర్శం
hash(key) % N	94.1% 5.9%

ఆదర్శం 5.9% • వాస్తవం 94.1% – 16 రెట్లు ఎక్కువ

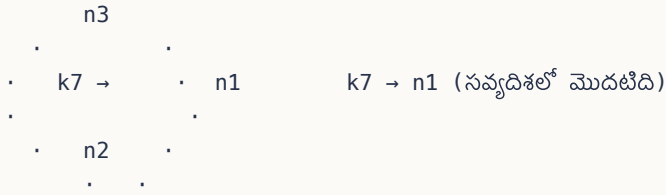
ఆదర్శం ఎందుకు 5.9%? కొత్త node కి తన వాటా (1/17) రావాలి, అంతే. మిగతా 16 nodes ఒకదానితో ఒకటి ఏమీ మార్పుకోసం లేదు.

కానీ % 17 అనేది % 16 తో ఏ సంబంధమూ లేని ఒక కొత్త function. కాబట్టి దాదాపు ప్రతి key కొత్త చోటికి వెళ్తుంది.

100 TB తో ఇది 94 TB ని network మీద తరలించడం – ఒక్క machine కలిపినందుకు. §1 ప్రకారం అది ప్రతి కొన్ని రోజులకీ జరుగుతుంది.

5. Step – ring · మరియు ring యొక్క సొంత విరుపు

Consistent hashing: keys మరియు nodes రెండింటినీ ఒకే వృత్తం మీద ఉంచడం. ఒక key తన నుంచి సవ్యదిశలో ఉన్న మొదటి node కి చెందుతుంది.



ఒక node కలిపితే అది **ఒకే ఒక arc** ని తీసుకుంటుంది – మిగతా అన్నీ అలాగే ఉంటాయి.

ring · 1 tokens/node | 1.4% | 5.9%

94.1% → 1.4%. ఇది ఆదర్శం (5.9%) కంటే కూడా తక్కువ. విజయం, అనిపిస్తుంది.

కాదు. 1.4% అనేది ఒక విజయం కాదు, ఒక లక్షణం

కొత్త node ఆదర్శం కంటే **తక్కువ** keys తీసుకుందంటే – దానికి **న్యాయమైన వాటా రాలేదు** అని అర్థం.

అంటే ring మీద nodes సమానంగా పంచబడలేదు. అదే ఇప్పుడు కొలుద్దాం.

2. కానీ ring సమానంగా పంచుతుందా? · 16 nodes · 2 లక్షల keys

tokens/node	కనిష్ట node	గరిష్ట node	గరిష్టం ÷ కనిష్టం	గరిష్టం ÷ సగటు
1	822	28,199	34.31×	2.26×

ఒక NODE కి 822 KEYS · ఇంకొక దానికి 28,199 · 34 రెట్లు

16 nodes ని యాదృచ్ఛికంగా ఒక వృత్తం మీద పడేస్తే, వాటి మధ్య arcs సమానంగా ఉండవు – కొన్ని పెద్దవి, కొన్ని సూక్ష్మమైనవి. ఇది గణితం, bug కాదు.

పరిణామం: ఒక node **34 రెట్లు** ఎక్కువ data, 34 రెట్లు ఎక్కువ traffic. అదే మీ p99. మిగతా 15 machines దాదాపు ఖాళీ.

మేము rebalancing సమస్యని పరిష్కరించి, ఒక load సమస్యని సృష్టించాం.

6. Step – virtual nodes

పరిష్కారం: ప్రతి భౌతిక node ని ring మీద **ఒక చోట కాదు, చాలా చోట్ల** ఉంచడం.

```
for (let v = 0; v < this.vnodes; v++)
  this.#tokens.push({ pos: h32(`${node}#${v}`), node });
```

128 tokens అంటే 16 nodes కి 2,048 arcs. పెద్ద సంఖ్యలో యాదృచ్ఛిక arcs **సగటు వైపు కుంచించుకుంటాయి.**

tokens/node	కనిష్ట node	గరిష్ట node	గరిష్టం ÷ కనిష్టం	గరిష్టం ÷ సగటు
1	822	28,199	34.31×	2.26×
4	4,200	30,653	7.30×	2.45×
16	6,959	18,649	2.68×	1.49×
64	10,153	14,788	1.46×	1.18×
128	10,146	14,672	1.45×	1.17×
256	10,564	14,189	1.34×	1.14×

మరియు ఇప్పుడు కదలికని మళ్ళీ చూద్దాం:

hash(key) % N	94.1%	5.9%
ring · 1 tokens/node	1.4%	5.9%
ring · 128 tokens/node	7.2%	5.9%

128 TOKENS ఎక్కువ KEYS ని కదిలించింది – మరియు అదే సరైనది

1 token: 1.4% కదిలాయి, అసమతుల్యత 34×

128 tokens: 7.2% కదిలాయి, అసమతుల్యత 1.45×

7.2% అనేది ఆదర్శం 5.9% కి **దగ్గరగా** ఉంది – కొత్త node తన న్యాయమైన వాటా తీసుకుంటోంది.

తక్కువ కదలిక ఎప్పుడూ మంచిది కాదు. సరైన కదలిక మంచిది. §4 lo 94.1% చాలా ఎక్కువ; §5 lo 1.4% చాలా తక్కువ. రెండూ అదే కారణానికి తప్పు – **వాటా న్యాయంగా లేదు.**

మరియు 128 తర్వాత లాభం ఆగిపోతుంది: 128 → 256 అసమతుల్యతని 1.45 నుంచి 1.34 కి మాత్రమే తగ్గిస్తుంది, మరియు metadata రెట్టింపు అవుతుంది.

Part 3 – రెండో విరుపు: ఇద్దరు ఒకేసారి రాస్తే

7. Step – రెండు clients, రెండు replicas

"ఎప్పుడూ రాయగలగాలి" అనే అవసరం ఒక పరిణామాన్ని బలవంతం చేస్తుంది: **ఏ replica అయినా ఒక రాతని స్వీకరించగలగాలి**, ఒక leader కోసం వేచి ఉండకుండా.

కాబట్టి ఇద్దరు clients ఒకే key ని వేర్వేరు replicas మీద ఒకేసారి రాయగలరు. ఆపై replicas ఒకరితో ఒకరు కలుస్తారు, మరియు ఎవరో ఒకరు నిర్ణయించాలి.

సహజమైన నియమం: **timestamp పెద్దది గెలుస్తుంది** – last-write-wins.

ఇప్పుడు ఒక shopping cart తీసుకుందాం. ముగ్గురు వాడుకరులు (లేదా ఒకే వాడుకరి మూడు tabs లో) **వేర్వేరు వస్తువులను** ఏకకాలంలో చేరుస్తారు. సరైన జవాబు స్పష్టం: **మూడూ cart లో ఉండాలి**.

8. రెండో విరుపు – LWW మూడింట రెండు పోగొడుతుంది

ప్రయోగం A – ముగ్గురు ఏకకాలంలో ***వేర్వేరు*** వస్తువులను చేరుస్తారు (తొలగింపులు లేవు)
సరైన జవాబు: **మూడూ ఉండాలి**.

వ్యాసం | గడియార తేడా | 500 ప్రయోగాల్లో పోయిన చేర్పులు | ఒక్క cart కి

lww		0 ms		1,000		2.00
lww		50 ms		1,000		2.00
vc		0 ms		0		0.00
vc		50 ms		0		0.00
orset		0 ms		0		0.00
orset		50 ms		0		0.00

ఒక్క CART కి సరిగ్గా 2.00 చేర్పులు పోయాయి – మూడింట రెండు

ఇది "అప్పుడప్పుడూ ఒక conflict" కాదు. LWW ఒక రాతని ఎన్నుకుని **మిగతా అన్నిటినీ పడేస్తుంది** – ఎందుకంటే అది మొత్తం విలువని ఒకే unit గా చూస్తుంది.

ముగ్గురు వాడుకరులు మూడు వేర్వేరు వస్తువులను చేర్చారు. Database ఒకదాన్ని ఉంచింది. **ఎవరికీ ఒక దేశం కనిపించలేదు.**

మరియు LWW గురించి ఒక ఇంకా చెడ్డ విషయం ఉంది – **అది తప్పు రాతని ఎన్నుకోగలదు**. రెండు వరుస రాతలు (ఒకటి తర్వాత ఒకటి, నిజ సమయంలో స్పష్టంగా), కానీ వేర్వేరు replicas మీద, వేర్వేరు గడియారాలతో:

ప్రయోగం C – రెండు వరుస రాతలు (ఒకదాని తర్వాత ఒకటి), వేర్వేరు replicas మీద.
తర్వాత జరిగిన రాత గెలవాలి. 1,00,000 ప్రయోగాలు.

గడియార తేడా (విస్తృతి) | తప్పు రాత గెలిచినవి | %

± 0 ms	0	0.0%
± 5 ms	822	0.8%
± 20 ms	4,552	4.6%
± 50 ms	11,967	12.0%
± 200 ms	37,396	37.4%
± 1000 ms	47,486	47.5%

±50 MS గడియార తేడాతో 12% రాతలు తప్పు క్రమంలో పరిష్కరించబడతాయి

±50 ms అనేది ఒక విపత్తు కాదు – అది **NTP ఉన్న ఒక సాధారణ datacenter**. ±1 సెకనుతో అది ఒక నాణెం విసరడం (47.5%).

అంటే LWW "తర్వాతిది గెలుస్తుంది" కాదు. అది "ఏ గడియారం ముందుకీ ఉండో ఆ replica గెలుస్తుంది".

Timestamp ఒక కారణ సంబంధాన్ని (causality) సూచించదు. అది ఒక అభిప్రాయాన్ని సూచిస్తుంది.

9. Step – vector clocks · మరియు తిరిగి వచ్చే వస్తువు

పరిష్కారం: గడియారాలు కాదు, **కారణ సంబంధం**. ప్రతి రాత ఒక vector clock మోస్తుంది – "నేను రాసేటప్పుడు ఏమేమి చూశాను".

```
const vcLeq = (a, b) => Object.keys(a).every(k => (a[k] || 0) <= (b[k] || 0));
const vcConc = (a, b) => !vcLeq(a,b) && !vcLeq(b,a); // ఏకకాలం
```

ఒక రాత ఇంకో దాన్ని **చూసి ఉంటే** అది దాన్ని మింగేస్తుంది. **చూడకపోతే** రెండూ ఉంటాయి – **siblings**. §8 యొక్క పట్టికలో **vc** వరుస సున్నా, ఎందుకంటే ఏ చేర్పు పడేయబడదు.

ఇప్పుడు ఒక వేరే పరిస్థితి, మరియు ఇదే Dynamo paper యొక్క ప్రసిద్ధ ఉదాహరణ:

Cart lo X మరియు Y ఉన్నాయి. వాడుకరి **X ని తొలగిస్తాడు** (replica 0 మీద). అదే సమయంలో ఇంకో tab (replica 1, ఇంకా X ఉన్న view తో) **Z ని చేరుస్తుంది**. సరైన జవాబు: **X పోవాలి. Y మరియు Z ఉండాలి.**

వ్యాసం · ఏ క్రమంలో చేరాయి	X తిరిగి వచ్చిందా	Z పోయిందా	Y పోయిందా
lww · తొలగింపు ముందు వస్తే	అవును ←	లేదు	లేదు
lww · చేర్పు ముందు వస్తే	లేదు	అవును ←	లేదు
vc · తొలగింపు ముందు వస్తే	అవును ←	లేదు	లేదు
vc · చేర్పు ముందు వస్తే	అవును ←	లేదు	లేదు

orset · తొలగింపు ముందు వస్తే		లేదు		లేదు		లేదు
orset · చేర్పు ముందు వస్తే		లేదు		లేదు		లేదు

VECTOR CLOCKS రెండు క్రమాల్లోనూ X ని తిరిగి తెచ్చాయి

ఎందుకంటే vector clocks "ఇవి రెండూ ఏకకాలం" అని **సరిగ్గానే** చెప్పాయి. ఆపై siblings ని కలపడానికి మనం వాడిన నియమం – **union** – X ని తిరిగి తెచ్చింది.

Vector clock తప్పు చేయలేదు. **Merge నియమం తప్పు.**

మరియు LWW ని గమనించండి: ఒక క్రమంలో X తిరిగి వస్తుంది, ఇంకో క్రమంలో Z పోతుంది. **ఏ సందేశం చివరగా చేరిందో దాన్ని బట్టి.** అది తప్పు జవాబు కాదు – అది **ఏకపక్షమైన** జవాబు, మరియు అది ఇంకా చెడ్డది.

OR-Set (observed-remove set) ఇదే పరిష్కరిస్తుంది: ప్రతి చేర్పుకీ ఒక ప్రత్యేక tag, మరియు ఒక తొలగింపు తను చూసిన tags ని మాత్రమే తొలగిస్తుంది.

```
add X → tag t1
add Y → tag t2
rem X → "నేను t1 ని చూశాను, దాన్ని తొలగించు"
add Z → tag t3      (t1 గురించి ఏమీ చెప్పదు)

merge: adds {X:t1, Y:t2, Z:t3} – removed {t1} = {Y, Z} ✓
```

ఇక్కడ అసలు పాఠం – మరియు ఇది **INTERVIEW LO** చాలా అరుదుగా వినిపిస్తుంది

Vector clocks ఒక conflict resolution వ్యూహం కాదు. అవి ఒక conflict *గుర్తింపు* యంత్రం.

అవి "ఈ రెండూ ఏకకాలం" అని చెబుతాయి. "అప్పుడు ఏమి చేయాలి" అని **చెప్పలేవు** – ఎందుకంటే జవాబు **data ఏమిటో** దాని మీద ఆధారపడుతుంది:

Cart → OR-Set. **Counter** → కూడిక. **Profile పేరు** → వాడుకరిని అడగడం. **Log** → రెండూ ఉంచడం.

కాబట్టి సరైన design ఇది: **database ఏకకాలాన్ని గుర్తించి siblings ని తిరిగి ఇస్తుంది; ఏమి చేయాలో application నిర్ణయిస్తుంది.** అదే Dynamo చేసేది, మరియు అదే దాన్ని వాడటం కష్టం చేసేది.

Part 4 – మూడో విరుపు: quorum

10. Step – $R + W > N$

ఇప్పుడు స్థిరత్వం. N కాపీలు, ఒక రాతకి W acks, ఒక చదవడానికి R జవాబులు.

$$N = 3, R = 2, W = 2 \rightarrow R + W = 4 > 3$$

తర్కం చక్కనిది: ఏ 2 nodes మరియు ఏ 2 nodes కి కనీసం ఒక ఉమ్మడి node ఉంటుంది. కాబట్టి ప్రతి చదవడం తాజా రాతని చూసిన కనీసం ఒక replica ని తాకుతుంది.

ఇది నిజం – మరియు అది ఒక షరతు మీద ఆధారపడుతుంది: ఆ 2 nodes నిజంగా అందుబాటులో ఉండాలి. §1 యొక్క పట్టిక ప్రకారం అది ఎప్పుడూ నిజం కాదు.

11. మూడో విరుపు – strict అయితే తిరస్కరణ, sloppy అయితే పాతది

12 nodes, nodes పడుతూ లేస్తూ ఉంటాయి, 12 seeds $\times$ 60,000 ticks:

$$N=3 \cdot R=2 \cdot W=2 \rightarrow R+W > N \cdot 12 \text{ nodes} \cdot \text{nodes పడతాయి, లేస్తాయి}$$

quorum రకం	రాతలు	తిరస్కరించిన రాతలు	తిరస్కరణ %	పాత చదవడాలు	పాత %
strict	2,87,943	71,503	19.89%	0	0.00%
sloppy	3,59,446	0	0.00%	1,302	0.45%

STRICT: 19.89% రాతలు తిరస్కరించబడ్డాయి • SLOPPY: 0.45% చదవడాలు పాతవి

Strict quorum అంటే "preference జాబితాలోని 3 nodes లో 2 ఉండాలి, లేకపోతే కాదు". స్థిరత్వం పరిపూర్ణం – 0 పాత చదవడాలు. మరియు cart **అయిదింటి ఒక్కసారి** "ఇప్పుడు కుదరదు" అంటుంది.

Sloppy quorum అంటే "3 nodes లో ఒకటి లేకపోతే, ring మీద తర్వాతి ఆరోగ్యకరమైన node కి రాయి". రాత ఎప్పుడూ విజయవంతం. కానీ ఆ రాత **preference జాబితాలో లేని** ఒక node మీద ఉంది – కాబట్టి తర్వాతి చదవడం దాన్ని **పూర్తిగా మిస్** కావచ్చు.

$R+W > N$ అనే అసమానత ఇక్కడ విరుగుతుంది – ఎందుకంటే "N" ఇక అదే N కాదు.

ఈ 19.89% నా SIMULATION యొక్క CHURN మీద ఆధారపడినది - దాన్ని అలాగే ఉపయోగించకండి

నా simulation lo nodes చాలా తరచుగా పడతాయి (tick కి 0.4%), ఎందుకంటే యంత్రాంగాన్ని కనిపించేలా చేయాలి.

§1 యొక్క పట్టికే నిజమైన సంఖ్య: ఒక node 1% సమయం అందుబాటులో లేకపోతే, రెండు replicas ఒకేసారి లేకపోవడం **0.03%**. అదే మీ strict-quorum తిరస్కరణ రేటు.

నా పట్టిక దిశని చూపుతుంది, పరిమాణాన్ని కాదు. పరిమాణం మీ fleet యొక్క ఆరోగ్యం మీద ఆధారపడుతుంది - మరియు అది మీరు కొలవాల్సిన సంఖ్య.

12. Step – hinted handoff, read repair • మరియు అవి ఎంత సరిపోవు

Sloppy quorum ఒక్కటే సరిపోదు – ఆ రాత తన సరైన ఇంటికి చేరాలి. **Hinted handoff**: ప్రత్యామ్నాయ node ఆ రాతని "ఇది నిజానికి n7 దే" అనే సూచనతో ఉంచుతుంది, మరియు n7 తిరిగి రాగానే పంపిస్తుంది.

hinted handoff – hint పట్టుకున్న node కూడా చనిపోతే?

hint holder చనిపోయే అవకాశం	సూచనలు	ఎప్పటికీ చేరని hints	పూర్తిగా పోయిన రాతలు
లేదు	79,973	0	0
0.02% / tick	79,942	2,828	14
0.10% / tick	80,627	11,465	184
0.50% / tick	77,526	31,311	1,288

చివరి నిలుపు వరుస – ఆమోదించిన రాతలు, ఏ ఇంటి **REPLICA** మీదా కాపీ లేకుండా

Hint పట్టుకున్న node చనిపోతే ఆ hint ఎప్పటికీ చేరదు. Preference జాబితాలో ఒక్క కాపీ కూడా లేకపోతే, ఆ రాత శాశ్వతంగా పోయింది – మరియు client కి "సరే" అని చెప్పాం.

184 అనేది 3.6 లక్షల రాతల్లో చిన్న సంఖ్య. కానీ అది **సున్నా కాదు**, మరియు అది **మౌనమైనది**.

ఇదే "ఎప్పుడూ రాయగలగాలి" అనే హామీ యొక్క నిజమైన ధర.

రెండో యంత్రాంగం: **read repair** – ఒక చదవడం జరిగినప్పుడు, వెనకబడిన replicas ని సరిచేయడం.

```
readRepair(key) {
  const r = this.get(key);
  for (const n of this.#liveOf(key)) {
    let merged = this.#node(n).get(key) || [];
    for (const v of r.versions) merged = this.#merge(merged, v);
    this.#node(n).set(key, merged);
  }
}
```

read repair ఉంటే/లేకపోతే – replicas ఎంత దగ్గరగా ఉంటాయి?

9 nodes · 4,000 steps · nodes పడతాయి లేస్తాయి · 40 runs · చివర్లో అందరూ ఫైకి

read repair | మూడు కాపీలూ ఒకేలా ఉన్న keys | వెనకబడిన కాపీలు

లేదు	1,028 / 2,000 (51%)	1,209
ఉంది	1,354 / 2,000 (68%)	807

51% → 68%. మెరుగుదల నిజమైనది, మరియు అది 100% కాదు

Read repair ఒక్కటే ఎప్పటికీ సరిపోదు, మరియు కారణం దాని నిర్వచనంలోనే ఉంది: **అది చదివిన keys ని మాత్రమే సరిచేస్తుంది.**

ఎవరూ చదవని ఒక key యొక్క వెనకబడిన కాపీ **శాశ్వతంగా** వెనకబడే ఉంటుంది. మరియు నిజమైన systems lo చాలా keys అరుదుగా చదవబడతాయి.

అందుకే మూడో యంత్రాంగం అవసరం – **anti-entropy**, replicas ఒకరితో ఒకరు తమ మొత్తం data ని పోల్చుకోవడం, ఎవరూ అడగకుండానే. Merkle trees దానికే (§16).

Part 5 – నిరూపణ మరియు ప్రదర్శన

13. మొత్తం code • 1.9 లక్షల రాతలు • mutation testing

ఏడు నియమాలు: **preference జాబితా ఎప్పుడూ N వేర్వేరు భౌతిక nodes, siblings ఎప్పుడూ నిజంగా ఏకకాలం, $R+W>N$ హామీ** (ఆ రాత తర్వాత దాని replicas ఏవీ పడకపోతే), **node కలిపితే గరిష్టంగా ఒక్క సభ్యుడే మారాలి, monotonic reads, keys సమానంగా పంచాలి, preference స్థిరంగా ఉండాలి.**

300 యాదృచ్ఛిక runs • 1,90,356 రాతలు • 1,68,905 చదవడాలు
నియమ ఉల్లంఘనలు: 0

ఏ దారులు నడిచాయి:

puts	1,90,356
gets	1,68,905
rejW	12,361
rejR	11,139
siblings	52,910
repairs	3,035
downs	6,967
conc	42,172

అక్కడికి చేరే ముందు fuzz నా code lo ఒక నిజమైన bug ని పట్టుకుంది:

ఆమోదించిన ఒక రాత, మౌనంగా మింగబడింది – 300 LO 8 SESSIONS

నా మొదటి vcBump ఇలా ఉండేది:

```
const vcBump = (c, id) => { const o = {...c}; o[id] = (o[id]||0) + 1; return o; };
```

అంటే counter **client యొక్క context** నుంచి వస్తుంది. ఖాళీ context తో ఇద్దరు clients అదే coordinator కి రాస్తే – ఇద్దరికీ సరిగ్గా ఒకే clock {n2: 1} .

ఆపై merge తర్వాత రెండేదాన్ని "పాతది" అని భావించి పడేసింది, మరియు put ఇప్పటికే "సరే" అని చెప్పేసింది.

సరైనది: counter **node యొక్క సొంతది**, monotonic, client context నుంచి కాదు:

```
this.counter.set(id, (this.counter.get(id)||0) + 1);  
const ver = { val, clock: { ...context, [id]: this.counter.get(id) } };
```

ఆపై fuzz నా పరీక్షల్లో ఒక తప్పుని కూడా పట్టుకుంది, మరియు అది ఇంకా విద్యాదాయకమైనది:

"ఏకకాల రాత" అని నేను పిలిచినది ఏకకాలం కాదు - అది ఒక ఉద్దేశపూర్వక OVERWRITE

నా harness ఖాళీ context తో **అదే coordinator** ద్వారా ఒక రాతని పంపి దాన్ని "ఏకకాలం" అని పిలిచింది. 300 lo 127 విఫలమయ్యాయి.

కానీ అదే coordinator ఆ రెండు రాతలనూ **వరుసగా** చూశాడు - కాబట్టి రెండోది మొదటిదాన్ని మింగడం **సరైనది**. ఖాళీ context అంటే "నాకేమీ తెలియదు, రాసెయ్" - అది Dynamo lo ఒక చెల్లుబాటు అయ్యే ఆపరేషన్.

నిజమైన ఏకకాలం కావాలంటే **వేరే coordinator** కావాలి. అలా మార్చాక - 0/300.

"ఏకకాలం" అనేది రెండు రాతలు ఒకే క్షణంలో జరగడం కాదు. అది ఒకరికొకరు తెలియకపోవడం.

==== mutation testing ====

మార్పు లేని code	→	0/300 విఫలం
preference lo నకిలీ భౌతిక nodes ని అనుమతిస్తే	→	120/120 విఫలం
ఉదా: k18: preference lo నకిలీ node		
ఒక్క node కి 128 tokens కి బదులు 1	→	120/120 విఫలం
ఉదా: అసమతుల్యం: గరిష్ట node 3288, సగటు 1538		
vector clock కి బదులు last-write-wins	→	63/120 విఫలం
ఉదా: k37: ఆమోదించిన v397 కనిపించలేదు		
కొత్తది పాతదాన్ని మింగే తనిఖీ తీసేస్తే	→	43/120 విఫలం
ఉదా: k54: ఆమోదించిన v517 కనిపించలేదు		
W తనిఖీ తీసేస్తే (ఒక్క కాపీ చాలు అంటే)	→	11/120 విఫలం
ఉదా: k10: ఆమోదించిన v193 కనిపించలేదు		
R nodes కి బదులు ఒక్క node నుంచే చదివితే	→	65/120 విఫలం
ఉదా: k14: ఆమోదించిన v388 కనిపించలేదు		
read repair తీసేస్తే	→	0/120 విఫలం

మరియు నా మొదటి MUTATIONS LO మూడు ఏమీ మార్చలేదు

మొదట నేను ఇలా మార్చాను: `constructor(vnodes = 128) → vnodes = 1, మరియు { N:3, R:2, W:2 } → W:1`.

మూడూ **0/120** చూపించాయి, మరియు నేను దాదాపు "నా పరీక్షలు బలహీనమైనవి" అని రాయబోయాను.

నిజమైన కారణం: నా harness ఆ విలువలన్నిటినీ **స్పష్టంగా ఇస్తుంది** - `new Ring(128), new Store(ring, {N:3, R:2, W:2})`. కాబట్టి default ని మార్చడం **ఏ code నీ మార్చలేదు**.

Logic నే మార్చాక (`v < this.vnodes → v < 1, live.length < this.W → < 1`) - **120/120, 11/120, 65/120**.

పాఠం: ఒక mutation బతికినప్పుడు, ముందు అది నిజంగా ఏదైనా మార్చిందా అని చూడండి.

చివరి మార్పు - **read repair తీసేస్తే 0/120** - నిజంగా బతికింది, మరియు అది సరైనదే: read repair ఒక **భద్రతా** ఆస్తి కాదు, ఒక **అభిసరణ** ఆస్తి. దాన్ని తీసేస్తే ఏదీ తప్పు జరగదు - విషయాలు **కలవడానికి ఎక్కువ సమయం** పడుతుంది. దాని సాక్ష్యం §12 యొక్క పట్టిక (51% → 68%), ఒక నియమం కాదు.

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

నిమిషాలు	ఏమి చెప్పాలి	ఏ సంఖ్య చెప్పాలి
0-5	Requirements · "ఎప్పుడూ రాయగలగాలి" అని స్థిరపరచడం	200 nodes · 1% unavailable → 2.97% requests కి ఒక replica తక్కువ
5-10	hash % N → rebalancing	94.1% keys కదులుతాయి
10-16	Ring → దాని సొంత విరుపు → vnodes	1.4% (తప్పు) · అసమతుల్యత 34.31x · 128 tokens → 1.45x
16-24	ఏకకాల రాతలు → LWW	ఒక్కో cart కి 2.00 చేర్పులు పోయాయి · ±50 ms → 12% తప్పు
24-32	Vector clocks → తిరిగి వచ్చే వస్తువు → OR-Set	vc రెండు క్రమాల్లోనూ X ని తిరిగి తెచ్చింది
32-40	R+W>N → strict vs sloppy	19.89% తిరస్కరణ vs 0.45% పాతవి
40-45	Hinted handoff, read repair, anti-entropy	184 పూర్తిగా పోయిన రాతలు · repair 51%→ 68%

ఈ **PROBLEM LO** మమ్మల్ని వేరుగా నిలబెట్టే ఒక్క క్షణం

చాలామంది "conflicts కి vector clocks వాడతాను" అని చెప్పి ముందుకి వెళ్తారు.

మీరు ఒక అడుగు ముందుకి: "**Vector clocks ఒక resolution వ్యూహం కాదు – అవి ఒక detection యంత్రం. నేను పరీక్షించినప్పుడు, vector clocks ఏకకాలాన్ని సరిగ్గానే గుర్తించాయి, ఆపై నా union merge తొలగించిన వస్తువుని తిరిగి తెచ్చింది. Vector clock తప్పు చేయలేదు – merge నియమం తప్పు. Cart కి సరైన సమాధానం OR-Set, counter కి కూడిక, profile పేరుకి వాడుకరిని అడగడం. Database ఏకకాలాన్ని చెబుతుంది; ఏమి చేయాలో application నిర్ణయిస్తుంది.**"

ఇది ఒక పేరు తెలుసని కాదు – ఆ యంత్రాంగపు పరిమితి ఎక్కడో మీరు చూశారని చూపుతుంది.

15. నోటితో చెప్పాల్సిన English script

Scope గురించి:

"This is the one system where horizontal scaling is genuinely easy – no joins, no transactions, no ordering between keys, so doubling the nodes doubles the throughput. That tells me the design budget goes somewhere else. With two hundred nodes and each node unavailable even one percent of the time, three percent of requests find one of their three replicas missing. Failure isn't an event here, it's the steady state."

Placement గురించి:

"Modulo-N placement moves ninety-four percent of keys when you add one node, against an ideal of six. Consistent hashing takes that to one point four percent – and I'd flag that as a warning sign rather than a win, because it's *below* the ideal, which means the new node didn't take its fair share. When I measured load with one token per node, the busiest node had thirty-four times the keys of the quietest. Virtual nodes fix it: a hundred twenty-eight tokens per node brings the imbalance to one point four five times, and now movement is seven percent – slightly above ideal, which is what fair looks like."

Conflicts గురించి:

"Because any replica must accept a write, two clients can write the same key concurrently. With last-write-wins I measured exactly two of every three concurrent cart additions silently discarded. It's worse than arbitrary, too: with fifty milliseconds of clock skew – which is a normal NTP datacenter – twelve percent of *sequential* writes resolve in the wrong order, because a timestamp records an opinion, not causality. Vector clocks fix detection. But when I ran the classic cart case – one tab removes an item while another adds a different one – vector clocks plus a union merge brought the removed item back, in both arrival orders. The clock was right; the merge rule was wrong. An observed-remove set gets it right, and the general principle is that the database detects concurrency and the application resolves it."

Quorums గురించి:

" R plus W greater than N guarantees overlap only if those nodes are actually reachable. In my simulation a strict quorum rejected about twenty percent of writes and served zero stale reads; a sloppy quorum rejected nothing and served zero point four five percent stale. I'd be careful with that twenty percent – my simulation is deliberately high-churn, and the real number follows from your node availability, so it's something to measure rather than quote. The subtler cost is hinted handoff: when the node holding a hint also dies, the write is gone from its home replicas entirely – a hundred eighty-four acknowledged writes in my run. And read repair only fixes keys somebody reads; it took full replication from fifty-one to sixty-eight percent, not a hundred, which is why you also need anti-entropy."

16. Follow-ups – Merkle trees, CRDTs, secondary indexes

"ఎవరూ చదవని keys ని ఎలా సరిచేస్తారు?"

Merkle trees. ప్రతి replica తన key పరిధిని ఒక hash వృక్షంగా నిర్మిస్తుంది. రెండు replicas పోల్చుకున్నప్పుడు – root hashes సమానం అయితే **అంతా సమానం, ఒక్క సందేహంలో**. వేరైతే, వేరుగా ఉన్న కొమ్మల్లోకి మాత్రమే దిగుతాయి. కాబట్టి ఖర్చు data పరిమాణానికి కాదు, **తేడాల సంఖ్యకి** అనులోమానుపాతంలో ఉంటుంది. **నేను దీన్ని అమలు చేయలేదు**, కాబట్టి దీనికి నా దగ్గర సంఖ్య లేదు – కానీ §12 యొక్క 68% ఎందుకు ఆగిపోయిందో అది వివరిస్తుంది.

"CRDTs అన్ని conflicts నీ పరిష్కరిస్తాయా?"

లేదు, మరియు ఇది ఒక ముఖ్యమైన మినహాయింపు. §9 యొక్క OR-Set ఒక CRDT, మరియు అది set semantics కి సరైనది. కానీ CRDT అనేది "ఏ merge అయినా సరైనది" అని కాదు – అది **"ఒక merge నియమం ఉంది, మరియు అది క్రమం మీద ఆధారపడదు"** అని. ఒక bank balance కి? "ఇద్దరూ ₹100 తీశారు, నిల్వ ₹150" అనే దానికి **గణితపరంగా సరైన merge లేదు** – అది ఒక వ్యాపార నిర్ణయం (overdraft ఇవ్వాలా, తిరస్కరించాలా). **CRDT ఆ నిర్ణయాన్ని తీసుకోదు.**

"Secondary indexes?"

ఇది Dynamo-తరహా stores lo అత్యంత కష్టమైన అభ్యర్థన, మరియు కారణం §1 lo ఉంది: keys మధ్య సంబంధం లేకపోవడమే scaling ని సులభం చేసింది. ఒక index అంటే **ఒక key ఇంకో key గురించి చెప్పడం** – అంటే ఆ స్వాతంత్ర్యాన్ని విరగొట్టడం. ఆచరణలో రెండు ఎంపికలు: **local index** (ఒక్కే partition lo, ప్రతి query అన్ని partitions కి scatter-gather) లేదా **global index** (ఇంకో table, asynchronously నవీకరించబడేది – అంటే **index పాతది కావచ్చు**, మరియు DynamoDB ఇదే చేస్తుంది).

"ఒక వేడి key ఉంటే?"

Virtual nodes సహాయపడవు – vnodes keys ని పంచుతాయి, ఒక్క key ని కాదు. ఒక key సెకనుకి లక్ష requests పొందితే, ఆ 3 replicas అదే మోయాలి. పరిష్కారాలు: ఆ key ముందు ఒక **cache**, లేదా key ని కృత్రిమంగా **విడగొట్టడం** (`cart-42#0` ... `cart-42#9` , చదివేటప్పుడు పది కలపడం) – మరియు ఆ రెండేది §9 యొక్క merge సమస్యని **పది రెట్లు** చేస్తుంది.

"ఈ doc lo నేను ఏమి కొలవలేదు"

మూడు విషయాలు, మరియు వాటి గురించి నా దగ్గర సంఖ్యలు లేవు: **Merkle tree anti-entropy, gossip ద్వారా membership వ్యాప్తి** (నా ring lo membership తక్షణమే అందరికీ తెలుసు – నిజమైన systems lo అది సెకన్లు పడుతుంది, మరియు ఆ కిటికీలో రెండు nodes వేర్వేరు preference జాబితాలను నమ్ముతాయి), మరియు **W యొక్క నిజమైన అసమకాలిక ప్రవర్తన** (నా `put` అన్ని live replicas కి ఏకకాలంలో రాస్తుంది, కాబట్టి W కేవలం ఒక అంగీకార పరిమితి).

17. ఏమి నేర్చుకున్నాం

1. Scaling సులభం అయితే, అది కష్టమైనది ఎక్కడో చెబుతోంది. Keys స్వతంత్రంగా ఉన్నాయి కాబట్టి nodes కలపడం సులభం – మరియు అందుకే 200 nodes ఉంటాయి, మరియు అందుకే **ఏదో ఒకటి ఎప్పుడూ పాడైపోయి ఉంటుంది.**

2. తక్కువ కదలిక ఎప్పుడూ మంచిది కాదు. % N 94.1% కదిలిస్తుంది (చాలా ఎక్కువ). 1-token ring 1.4% (చాలా తక్కువ). రెండూ అదే కారణానికి తప్పు – వాటా న్యాయంగా లేదు.

3. ప్రతి పరిష్కారానికి దాని సొంత విరుపు ఉంది. Ring rebalancing ని పరిష్కరించి **34 రెట్లు అసమతుల్యత** తెచ్చింది. Vector clocks LWW ని పరిష్కరించి **తొలగించిన వస్తువుని తిరిగి తెచ్చాయి.**

4. Timestamp ఒక కారణ సంబంధం కాదు, ఒక అభిప్రాయం. ± 50 ms తేడాతో **12% వరుస రాతలు** తప్పు క్రమంలో పరిష్కరించబడతాయి.

5. Vector clocks ఒక గుర్తింపు యంత్రం, ఒక పరిష్కార వ్యూహం కాదు. "ఇవి ఏకకాలం" అని చెప్పడం ఒక పని; "అప్పుడు ఏమి చేయాలి" అనేది **data రకాన్ని బట్టి**, మరియు database కి అది తెలియదు.

6. "ఏకకాలం" అంటే ఒకే క్షణం కాదు – ఒకరికొకరు తెలియకపోవడం. నా harness ఆ రెండింటినీ కలిపేసి **300 to 127** విఫలమైంది.

7. $R+W>N$ ఒక షరతు మీద ఆధారపడిన హామ్. ఆ nodes అందుబాటులో లేకపోతే మీరు ఒకటి ఎంచుకోవాలి: **19.89% తిరస్కరణ**, లేదా **0.45% పాత చదవడాలు.**

8. "ఎప్పుడూ రాయగలగాలి" కి ఒక మౌనమైన ధర ఉంది. Hint holder చనిపోతే ఆ ఆమోదించిన రాత పూర్తిగా పోతుంది – నా runs to 184.

9. ఒక mutation బతికితే, ముందు అది నిజంగా ఏదైనా మార్పిందా అని చూడండి. నా మూడు mutations defaults ని మార్చాయి, harness వాటిని override చేసింది – అవి **mutations కానే కాదు.**

ఈ **DOC** నుంచి ఒక్క వాక్యం గుర్తుపెట్టుకోవాలంటే

Dynamo ఒక database కాదు – అది ఒక వరుస వ్యాపారాల జాబితా, మరియు ప్రతి దాన్నీ మరే ఎంచుకోవాలి.

ఎన్ని vnodes (metadata vs సమతుల్యత). ఏ merge నియమం (సరళత vs సరైనతనం). Strict లేక sloppy (స్థిరత్వం vs లభ్యత). R మరియు W (వేగం vs భద్రత).

మిగతా systems మీ కోసం ఈ నిర్ణయాలు తీసుకుంటాయి. Dynamo **తీసుకోదు** – అది వాటిని settings గా బయటపెడుతుంది, మరియు ప్రతి setting వెనక ఈ doc to ఉన్న ఒక పట్టిక ఉంది.

అందుకే దాన్ని అర్థం చేసుకోవడం కష్టం, మరియు అందుకే అది ఆగదు.